DATV-Linux

DVB-S2 Transmitter User Manual

Linux GUI and CLI for amateur-radio DATV

Flexible multi-SDR operation for HackRF One, LimeSDR Mini, and ADALM Pluto / Pluto+

Works with stock ADALM Pluto / Pluto+ firmware; Pluto MQTT firmware support is optional

Audience	Platform	Scope
Radio amateurs / DATV operators	Linux desktop / Raspberry Pi 5	GUI operation, CLI use, and Pluto firmware modes

Created by Radio Amateur Lebanon (OD5RAL) Engineering Team

Figure 1. High-level DATV-Linux signal path.

What this manual covers. This guide explains installation, first-time setup, device selection, bitrate planning, Pluto stock and firmware-based operation, webcam and external stream input, stop/exit behavior, CLI examples, and troubleshooting.

Contents

- 1. Overview
- 2. Supported devices and Pluto operating modes
- 3. Installation and first launch
- 4. Quick-start workflows
- 5. Main tab reference
- 6. Encoder tab reference
- 7. Source / Overlay tab reference
- 8. Console, Pluto maintenance, and stop/exit policy
- 9. Bitrate planning and low symbol-rate use
- 10. Command-line operation
- 11. Troubleshooting
- 12. Good operating practice

> Important. DATV-Linux works with stock ADALM Pluto / Pluto+ firmware; F5OEO and F5UII integrations are optional. If you use a custom Pluto firmware, make sure it is updated to the latest version. Firmware-specific Pluto MQTT and pass-mode control belong to the GUI. The CLI remains the direct SDR backend path and does not replace the GUI's firmware handoff logic.

1. Overview

DATV-Linux is a Linux-based DVB-S2 transmitter for amateur-radio digital amateur television (DATV). It accepts video from a file, a webcam/camera, or an external FFmpeg-compatible input, turns that media into an MPEG transport stream, and feeds the transmitter core that builds DVB-S2 baseband frames, applies FEC and framing, shapes the waveform, and sends IQ to the selected SDR.

The software is aimed at practical station operation and flexibility. The GUI helps plan bitrate, keeps the transport stream inside the selected DVB-S2 mode as far as possible, shows live transmission statistics, and can either drive ordinary SDR backends directly or coordinate Pluto firmware paths when MQTT-based handoff is required.

Main advantages of DATV-Linux:

• One Linux workflow across several SDR devices, including HackRF One, LimeSDR Mini, stock ADALM Pluto / Pluto+, and optional Pluto firmware paths.

• Works with stock ADALM Pluto / Pluto+ manufacturer firmware through libiio; no custom Pluto firmware is required for normal Pluto operation.

• Keeps encoding, bitrate planning, and DVB-S2 generation on the Linux host, which makes tuning and troubleshooting easier.

This manual reflects the current public user release.

Area	What DATV-Linux provides
RF outputs	HackRF One, LimeSDR Mini via SoapySDR, ADALM Pluto / Pluto+, PlutoDVB2 F5OEO, and Pluto F5UII 0303/2402.
Source modes	Video File, Webcam / Camera, and External Stream (OBS / FFmpeg compatible input URL).
DVB-S2 controls	Modulation, FEC, frame size, symbol rate, roll-off, pilots, Gold code, bitrate estimate, and occupied RF bandwidth estimate.
Encoding controls	Profiles, codec selection, resolution, frame rate, preset, tune,

	video bitrate share, GOP size, audio codec/bitrate, and MPEG-TS service labels.
Operations tools	RF device detection, Pluto maintenance panel, transmission console, live statistics, and stop/exit behavior for Pluto-backed paths.
Flexibility	One Linux workflow across several SDR devices. Stock ADALM Pluto / Pluto+ works through libiio, and custom Pluto firmware support is optional rather than required.

2. Supported devices and Pluto operating modes

The RF output device is selected on the Main tab. The meaning of that choice matters most for Pluto, because DATV-Linux supports both direct libiio operation and firmware-specific MQTT control paths. The current public build uses host-side DVB-S2 generation for its normal transmit paths.

Pluto operating models in plain language:

Stock Pluto / direct IQ mode. DATV-Linux builds the DVB-S2 waveform on the Linux host and sends IQ or sample data to Pluto over libiio. This works with the manufacturer firmware on ADALM Pluto and Pluto+.

Pluto firmware pass/IQ mode. DATV-Linux still builds the DVB-S2 waveform on the Linux host. The GUI uses MQTT only to prepare the Pluto firmware, switch it to the expected pass mode, and keep the firmware-specific control path aligned during transmit.

Pluto internal TS mode (future concept, not the current public build). DATV-Linux would stop after MPEG-TS generation and pass TS plus control to Pluto, and Pluto would then perform the DVB-S2 stages internally.

GUI device	Transport path	What you must set	Best used for
HackRFone	Direct backend	Frequency, TX level, optional internal amp	Simple direct transmission and lab testing
LimeSDR mini	Direct SoapySDR backend	Frequency, TX level	Direct DATV transmit with Lime hardware
AdalmPluto	Direct libiio path	Pluto IP address, usually 192.168.2.1 when connected by USB	Stock/default Pluto or Pluto+ using manufacturer firmware
PlutoDVB2 F5OEO	GUI-assisted MQTT + pass-mode handoff	Pluto IP plus the exact station callsign/topic key configured in the firmware	Optional PlutoDVB2 / F5OEO firmware path using MQTT on TCP 1883
Pluto F5UII 0303 2402	GUI-assisted MQTT control path	Pluto IP; callsign is optional for overlays and labels	Optional 0303/2402 controller path using plutodvb/subvar topics on TCP 8883

> **Pluto note.** For PlutoDVB2 F5OEO, the station callsign field must match the firmware topic key exactly. If the callsign is wrong, MQTT control may fail even if the Pluto is reachable over the network.

Auto-detection is conservative. Stock/direct Pluto is identified from a confirmed Pluto-class IIO context. The firmware profiles are only marked as detected when the expected MQTT ports are reachable as well. HackRF and LimeSDR Mini are only marked as detected when their probe tools report a real device.

3. Installation and first launch

DATV-Linux is intended for Linux systems with FFmpeg, Python 3, and the relevant SDR development libraries and utilities installed. The one-shot installer is the easiest way to prepare a normal station machine.

3.1 Recommended install path

```
chmod +x install.sh
./install.sh
```

The installer builds the transmitter, installs the Python requirements, and places the main components where the GUI can find them. `cmake --install` also installs the GUI script, the dependency checker, the installer, and the support files used by the GUI.

3.2 Manual build

```
sudo apt install -y build-essential cmake pkg-config \
  libhackrf-dev hackrf \
  libiio-dev libiio-utils \
  libusb-1.0-0-dev \
  libsoapysdr-dev soapysdr-tools soapysdr-module-lms7 limesuite \
  ffmpeg python3 python3-pyqt6

cmake -S . -B build
cmake --build build -j$(nproc)
cp build/dvbs2_tx .
cmake --install build
```

3.3 Dependency check

```
python3 check_dependencies.py
```

The dependency checker verifies the compiler, FFmpeg, SDR utilities, Python modules, the `dvbs2_tx` binary, optional Pluto maintenance tools, and optional webcam/microphone tools such as `v4l2-ctl`, `pactl`, and `arecord`.

3.4 Starting the GUI

```
python3 dvbs2_gui.py
```

The application window opens first and heavy probing continues in the background. That means the GUI should become usable quickly even on slow or partly configured systems. Use Refresh Detection if you connect hardware after launch.

> **Window size presets.** The title bar area includes Raspberry Pi 5, Standard, and Large window presets. Raspberry Pi 5 is the smaller layout intended for compact displays; Standard is the normal desktop layout; Large opens the widest working view.

4. Quick-start workflows

The fastest way to learn DATV-Linux is to use a known-good starting point. These recipes assume normal QPSK DATV work and a reachable receiver for monitoring.

4.1 The basic workflow for any device

1. Enter your station callsign near the top of the window. DATV-Linux uses it for overlays and service labels, and it is mandatory for PlutoDVB2 F5OEO.
2. On the Main tab, select modulation, FEC, frame size, symbol rate, roll-off, and pilots.

3. Choose the RF device and fill in the required device-specific fields.
4. Choose a media source: Video File, Webcam / Camera, or External Stream.
5. Open the Encoder tab and either keep a profile or set the encoding controls manually.
6. Check the bitrate guidance box. If it shows Blocked, fix the bitrate plan before starting.
7. Press START. Monitor the receiver and the live statistics. Press STOP when finished.

4.2 Stock ADALM Pluto / Pluto+

8. Select AdalmPluto as the device. No custom Pluto firmware is required for this mode.
9. Enter the Pluto IP address, usually 192.168.2.1 when the Pluto is connected by USB.
10. Click Refresh Detection and confirm that AdalmPluto is marked detected.
11. Choose a source. For first tests, Video File plus Use Test Pattern is the quickest path.
12. Pick Auto Detect or Pi 5 Safe on the Encoder tab.
13. Start with a conservative DVB-S2 mode such as QPSK with pilots enabled. This is the recommended starting point if you want Pluto support without depending on F5OEO or F5UII firmware.

4.3 PlutoDVB2 F5OEO firmware

14. Select PlutoDVB2 F5OEO.
15. Enter the Pluto IP address and the exact station callsign/topic key configured in the firmware.
16. Confirm that libiio access works and that the firmware MQTT broker is reachable on TCP 1883.
17. Use the GUI to start transmission. The GUI handles the pass-mode handoff and MQTT context updates.
18. If you see strange behavior above 250 kS/s, verify that the Pluto firmware is updated to the latest version. In field use, outdated firmware was the main cause of several higher-rate failures.

4.4 Pluto F5UII 0303/2402

19. Select Pluto F5UII 0303 2402.
20. Enter the Pluto IP address.
21. Refresh Detection and confirm that the 0303/2402 path is detected.
22. Start transmission from the GUI. The GUI uses the documented PlutoDVB MQTT topic family on TCP 8883.

The firmware modes are optional. DATV-Linux remains fully usable with stock Pluto firmware and with other SDR devices such as HackRF One and LimeSDR Mini.

4.5 HackRF One and LimeSDR Mini

23. HackRF One: select HackRFone, set frequency and TX level, and enable the internal amp only if your setup needs it.
24. LimeSDR Mini: select LimeSDR mini, verify detection, and set frequency and TX level as usual.
25. For first bring-up with either device, use a video file or the bundled test pattern before moving to webcam or live external sources.

5. Main tab reference

5.1 DVB-S2 Physical Layer

This group defines the RF transport mode. The combination of modulation, FEC, frame size, symbol rate, roll-off, pilots, and Gold code determines the usable transport capacity and occupied bandwidth. In normal operation, keep Gold code at 0; non-zero values are reserved for future radio-amateur receiver upgrades.

Control	Purpose
Modulation	QPSK, 8PSK, 16APSK, or 32APSK.
FEC Rate	Coding rate available for the selected modulation and frame size.
Frame Size	Short (16200 bits) or Normal (64800 bits).

Symbol Rate	Carrier symbol rate in kS/s. You may choose a preset or type a value.
Roll-off	0.20, 0.25, or 0.35.
Enable Pilots	Recommended in most practical DATV operation.
Gold Code	Physical-layer scrambling code. Keep this at 0 (default) in normal operation; non-zero values are reserved for future radio-amateur receiver upgrades.
Bitrate estimate	Estimated usable DVB-S2 transport stream rate.
Bandwidth estimate	Estimated occupied RF bandwidth for the chosen parameters.

> **Starting point.** When you are unsure, start with QPSK, a conservative FEC, Normal frame, pilots on, and roll-off 0.20. Get stable lock first, then optimize.

5.2 RF Transmitter

This group selects the output hardware and the immediate RF parameters. The visible fields change slightly depending on the selected device.

- Device selects the SDR backend or Pluto firmware mode.
- Frequency is entered in MHz on the GUI.
- TX Level is shown in dB. For HackRF One, the normal range is 0 to 47 dB.
- Enable Internal Amp is HackRF-only.
- Pluto IP Address is used for stock Pluto and the two Pluto firmware paths.
- Refresh Detection re-runs the hardware probes.
- The local TX sample-rate estimate shows the planned baseband rate and SPS strategy.

Below the controls, DATV-Linux shows per-device status lines for LimeSDR mini, HackRFone, AdalmPluto, PlutoDVB2 F5OEO, and Pluto F5UII 0303 2402.

5.3 Media Source

The media source can be a local file, a Linux webcam/camera, or an external FFmpeg-compatible stream.

Source mode	How to use it
Video File	Browse to a local file, or click Use Test Pattern for the bundled DATV test clip.
Webcam / Camera	Select a V4L2 camera, optionally route microphone/line input, and use custom FFmpeg input strings only if the automatic probe is not enough.
External Stream (OBS / FFmpeg)	Enter an FFmpeg input URL such as UDP, SRT, or another supported live source.

5.4 Performance Statistics

The Main tab also shows live performance values while transmitting.

- Frames Sent shows how much payload has been pushed through the software chain.
- Buffer Level shows the transmitter ring buffer fill state.
- Frame Rate and Bitrate show the current production side of the stream.
- Underflows should remain at zero in a healthy run.

6. Encoder tab reference

The Encoder tab controls video, audio, and MPEG-TS service labeling. The guidance box on this page is one of the most important safety checks in the program.

6.1 Profiles

Profile	Intent
Auto Detect	Adapts to the estimated TS rate and host capability. Prefers a validated encoder after background probing completes.
Custom	Leaves the current manual settings unchanged.
Pi 5 Safe	Conservative software-oriented path for Raspberry Pi 5 class systems.
Linux PC Balanced	Balanced default for a normal desktop or laptop.
Low SR DATV	Conservative low-rate profile for narrow DATV channels.
Higher SR DATV	Balanced profile for roomier symbol-rate budgets.

Auto Detect, Pi 5 Safe, and Low SR DATV are smart profiles. At very low transport rates they can reduce the requested audio bitrate, or disable audio entirely, to keep the transport valid.

6.2 Low SR stabilization

Low SR stabilization (<250 kS/s) is optional and is off by default. Enable it only when you need extra hardening for very low symbol-rate work.

- It applies more conservative FFmpeg timing and queue settings.
- It adds extra transport-planning headroom in the GUI.
- It can enable a stronger low-symbol-rate pulse-shaping path in the transmitter core.

6.3 Video controls

- Resolution and FPS define the outgoing frame size and rate.
- Codec is either Auto or a specific FFmpeg encoder. Hardware encoders appear only if the probe confirms they are usable.
- Preset and Tune apply mainly to software encoders such as libx264 or libx265.
- Video Bitrate (% of net) reserves a share of the estimated TS rate for video.
- GOP Size controls keyframe spacing. A smaller GOP is usually safer for low-rate DATV.

6.4 Audio controls

- Enable Audio turns the audio path on or off.
- Bitrate and Codec set the audio contribution inside the TS budget.
- At very low TS rates, aggressive audio settings can block START because the transport would not fit.

6.5 MPEG-TS Service

Service Name and Service Provider are written into the transport stream so SDRangel and compatible receivers can show them as the MPEG-TS service information.

> **Read the color.** Green guidance means your current choice is within the conservative profile. Yellow means advisory: it can work, but you should verify on the real receiver. Red means blocked or clearly too aggressive for the current transport budget.

7. Source / Overlay tab reference

This tab combines the source-specific controls with the text overlay controls.

7.1 Overlay controls

- Enable Text Overlay turns the overlay on or off.
- Overlay Text lets you type the label to render into the video.
- If Overlay Text is blank, DATV-Linux can use the main station callsign automatically.
- Position, Font Size, and Font Color change how the overlay is drawn in FFmpeg.

7.2 Webcam / Camera details

When Webcam / Camera is selected as the source, extra controls appear for camera and microphone handling.

Control	Use
Refresh Cameras	Refresh the V4L2 camera list.
Refresh Mics	Refresh PulseAudio / PipeWire / ALSA microphone sources.
Route microphone / line input with webcam	Adds audio capture from the selected microphone route when audio is enabled.
Custom Video Input	Optional full FFmpeg input override for the camera, including -f and -i.
Custom Mic Input	Optional full FFmpeg input override for the microphone, including -f and -i.

> **Custom input rule.** Leave the custom input strings empty for automatic probing. Use them only when you need to override the automatic path. Each custom string must contain the full FFmpeg input arguments, including the input format and the -i source.

7.3 External stream input

External Stream accepts any FFmpeg-compatible input URL. The default example is a UDP listener, but SRT and other FFmpeg input formats can be used as well.

```
udp://0.0.0.0:23000?fifo_size=1000000&overrun_nonfatal=1
```

8. Console, Pluto maintenance, and stop/exit policy

The Console tab is the operator view during startup and transmission. It logs the build/run decisions, device control messages, and transmitter status.

8.1 Console use

- Read the top summary before each transmission. It shows the selected MODCOD, symbol rate, device, source, bitrate plan, and encoder choice.
- Watch the console for warnings about bitrate planning, missing source inputs, or failed device coordination.
- For Pluto firmware paths, the console is the place where you can confirm the MQTT handoff sequence and pass-mode activity.

8.2 Pluto maintenance panel

The public user release keeps Pluto maintenance intentionally simple. The panel shows:

- Context Model
- Refresh Pluto Info

- Reboot Pluto
- Stop / Exit Policy

Refresh Pluto Info reads the context model over libiio. Reboot Pluto uses the reboot helper path and therefore depends on working remote access to the Pluto.

8.3 Stop / Exit Policy

Policy	What happens
Pass only	Ends the local transmission and leaves the Pluto reachable. On Pluto firmware paths this means a final pass-mode request where supported.
Mute Pluto TX	Attempts to push the Pluto TX attenuation to a safe value after stopping.
Reboot Pluto	Stops the session and reboots the Pluto after transmission ends.

> **Closing the program.** For Pluto-backed devices, the selected stop/exit policy is applied both when you press STOP and when you close the GUI window. Even so, pressing STOP first is still the best habit because it lets you see the final console messages.

9. Bitrate planning and low symbol-rate use

The most common DATV failure is asking the encoder to produce more payload than the selected DVB-S2 mode can carry. DATV-Linux tries to prevent that with a bitrate estimate and a guidance box.

Use the guidance as an operating limit, not a decoration. When the guidance warns that the current resolution, frame rate, or audio plan is too aggressive, fix the plan before going on-air.

Approximate TS class	Conservative starting point	Audio guidance	Typical profile
Below ~90 kbps	320x240 at 10 fps	Usually video only	Low SR DATV or Auto Detect
~90 to 180 kbps	320x240 at 10–12 fps	24 kbps AAC-LC only if needed	Low SR DATV
~180 to 260 kbps	320x240 at 15 fps	24–32 kbps audio can fit	Low SR DATV / Pi 5 Safe
~260 to 420 kbps	480x270 at about 20 fps	32 kbps audio is usually reasonable	Auto Detect or Linux PC Balanced
~420 to 900 kbps	640x360 at 25 fps	Normal low-delay audio plans fit more easily	Linux PC Balanced

> **Sub-250 kS/s operation.** When you work below 250 kS/s, keep expectations conservative. Start with lower frame rates, smaller pictures, and simple content. Only enable Low SR stabilization if you really need the extra hardening.

If START is blocked because of bitrate, reduce resolution, reduce FPS, lower the audio bitrate, disable audio, or raise the symbol rate / choose a higher-capacity DVB-S2 mode.

10. Command-line operation

The CLI binary reads MPEG transport stream data from standard input and transmits it to the selected SDR backend. This is useful for scripted work and direct backend testing.

```
./dvbs2_tx <MODCOD> <FREQ_HZ> <SYMBOL_RATE> <TX_LEVEL> <AMP_ENABLE> [ROLLOFF] [PILOTS]
[GOLDCODE] [DEVICE] [DEVICE_ARG]
```

Example MODCODs include QPSK_1/2_S, QPSK_3/4_N, 8PSK_3/4_S, and 8PSK_5/6_N.

For Pluto firmware ecosystems, use the GUI for MQTT and pass-mode coordination. The CLI remains the direct backend path and does not replace the GUI handoff logic.

10.1 CLI examples

```
ffmpeg -re -i video.ts -f mpegts - | ./dvbs2_tx QPSK_1/2_S 2407500000 250000 35 1 0.20 1
0 hackrf
ffmpeg -re -i video.ts -f mpegts - | ./dvbs2_tx QPSK_1/2_S 2407500000 250000 10 0 0.20 1
0 pluto ip:192.168.2.1
ffmpeg -re -i video.ts -f mpegts - | ./dvbs2_tx QPSK_1/2_S 2407500000 250000 40 0 0.20 1
0 limesdrmini driver=lime
```

> GUI versus CLI. Make sure your firmware is updated to the latest version. Use the GUI when you need Pluto firmware-specific MQTT and pass-mode handling. The CLI is the direct backend path; it does not replace the GUI's firmware orchestration.

11. Troubleshooting

Symptom	Likely cause	What to do
START stays disabled	Missing source path, blocked bitrate plan, or incomplete Pluto firmware inputs	Check the guidance box, callsign field, source path, and device detection status.
PlutoDVB2 F5OEO will not start	Wrong station callsign/topic key, MQTT not reachable, or firmware not in the expected state	Confirm the callsign exactly matches firmware config and verify TCP 1883 reachability.
0303/2402 path not detected	MQTT 8883 not reachable or Pluto identity not confirmed	Verify the Pluto IP, libiio access, and that the 0303 broker is actually present on TCP 8883.
No receiver lock or distorted constellation	Wrong RF settings, too much drive, too aggressive bitrate, or outdated Pluto firmware	Reduce gain, confirm receiver settings, simplify the bitrate plan, and update old Pluto firmware if higher symbol rates misbehave.
Webcam does not appear	No V4L2 device, missing probe tools, or unusual FFmpeg capture requirements	Use Refresh Cameras, test with v4l2-ctl, or provide a custom FFmpeg webcam input string.
Microphone audio missing in webcam mode	Main Audio is off, mic route is off, or wrong input source	Enable Audio, enable Route microphone / line input with webcam, then select or override the mic source.
Reboot helper fails	Remote access to Pluto is not working	Verify that the Pluto is reachable and that remote reboot access is set up.

11.1 Pluto-specific field advice

- If a PlutoDVB2 F5OEO system fails above 250 kS/s but lower rates work, update the Pluto firmware before changing GUI settings.
- If stock Pluto is reachable but the firmware profiles are not detected, that is expected unless the correct MQTT port is also reachable.
- If both Pluto firmware MQTT ports appear reachable, treat that as an ambiguous setup and verify which controller is actually active on the Pluto.

12. Good operating practice

- Operate only within your amateur-radio licence conditions, local band plan, and the rules that apply in your region.
- For first tests, use the bundled test pattern or a known-good video clip before trying live camera or external feeds.
- Start with conservative DVB-S2 settings and low RF power, then work upward.
- Watch a real receiver or spectrum display while testing; do not assume that a running GUI automatically means a clean on-air signal.
- Keep notes of which profiles, symbol rates, and audio settings work on your exact hardware.

> **Final advice.** The cleanest results come from a simple first configuration: QPSK, pilots on, conservative audio, modest resolution, and a verified receiver. Get a robust lock first; optimize later.

Appendix: Useful commands

```
python3 check_dependencies.py
python3 dvbs2_gui.py
ffmpeg -re -i input.mp4 -f mpegts - | ./dvbs2_tx QPSK_1/2_S 2407500000 250000 35 1 0.20 1
0 hackrf
```